The primitives' page: the join and the visible write

Programming Paradigms · Lab 7 · session 7: multi-threading

Ten questions on the two primitives of the lecture that are not structures: the join, on which `Workers.join` rests, and the visible write, on which its stop flag rests. The sheet goes with the warm-up, for the week; the teacher's copy has the answers. The lab collects nothing.

Three properties recur. **Mutual exclusion**: no two threads are inside the code that changes shared data at the same time. **Ordering**: a thread acts only after another has acted. **Visibility**: a write on one thread is seen by a read on another. A lock, an atomic instruction and a copy per thread give mutual exclusion. A join, a barrier and a monitor's wait give ordering, and visibility with it. A volatile write gives visibility only.

The join

The guarantee.

The join. When `t.join()` returns, `t` has ended, and everything `t` wrote is visible to the thread that joined it. `t.start()` is the mirror: everything written before the start is visible to `t`.

Question 1

Given a worker that writes its result into `counts[id]` and a main thread that reads the array, say what main may read in `counts[id]` before the join, after the join, and after joining only the other workers.

Question 2

Given three workers that each write their own slot of `counts`, joined in the order 2, 0, 1, say whether the order of the joins changes what main sees after the three.

Question 3

Given two workers that both write the plain field `last`, both joined, say what the join guarantees about `last` and what it leaves open.

Question 4

Given a thread `t` created with `new Thread(r)` and never started, say when `t.join()` returns and what it guarantees about the writes of `r`.

Question 5

Given the join's guarantee, say which of the three properties it gives: mutual exclusion, ordering, visibility.

The visible write

The guarantee.

The visible write. All reads and writes of volatile fields fall in one order that every thread agrees on, consistent with each thread's own order. A read of a volatile field sees the last write to it in that order, and everything the writer wrote before that write is visible after the read. A volatile field does not make a read-modify-write one step, and does not make two fields change together.

Question 6

Given the loop `while (!stop) { }` on a plain field `stop`, set to `true` by another thread, say whether the loop may run for ever and which rule allows it; then the same with `stop` declared `volatile`. The third line of `WorkersCheck` runs this case.

Question 7

Given a plain field `data` written by thread A before A sets the volatile flag `ready` to `true`, and a thread B that reads `ready` as true, say what B may read in `data`.

Question 8

Given `volatile int n`, two threads that each run `n++` a thousand times, and `n` printed after both are joined, say whether the result is always 2000; if not, give its range and say why `volatile` does not prevent the loss.

Question 9

Given two volatile fields `x` and `y`, both 0, thread A running `x = 1; r1 = y` and thread B running `y = 1; r2 = x`, say whether `r1` and `r2` can both end at 0; then the same with plain fields. Then run `java -cp out Reorder plain` and `java -cp out Reorder volatile` (`warmup/Reorder.java`). A run that shows no reordering proves nothing (rule 8).

Question 10

Given four needs, a stop signal; a counter that many threads add to; a count that one thread writes and another reads after a join; two fields that must change together, say which tool fits each, a volatile flag, an atomic, a count per thread or a lock, and why the others do not.